Structural Econometrics and Inverse Reinforcement Learning: Inferring preferences and beliefs from human behavior*

John Rust† Pranjal Rawat‡

June 30, 2025

PRELIMINARY DRAFT: NOT FOR CIRCULATION

*John Rust is grateful for support from his Georgetown University Professorship fund.
†Professor, Georgetown University jf693@georgetown.edu
‡PhD Candidate Georgetown University. pp712@georgetown.edu

1 Introduction

We survey two related literatures: *dynamic discrete choice* (DDC) in structural econometrics (SE) and "inverse reinforcement learning" (IRL) in machine learning (ML). Both are focused on inferring preferences of individuals from observations of their choices over time assuming they are *rational* i.e. their choices maximize an expected reward ("utility") function based on beliefs of how their actions affect current and uncertain future states, outcomes, and payoffs, allowing for unobserved states ("noise") that result in decision rules that appear to be probabilistic from the standpoint of an outside observer who only partially observes agents' states, Rust (1987). The literatures differ in the methods used to solve the decision problem and infer agents' preferences, and have different rationales for inferring/learning them.

SE focuses on testing and evaluating economic theories and making *counterfactual predictions* of the effect of changes in the environment and economic policies on economic behavior, outcomes, and welfare.[1] Structural models differ from reduced-form models that summarize behavior without inferring preferences. Reduced-form models require historical policy variation for counterfactuals, but cannot forecast unprecedented policies. Structural models simulate how agents would react to new policies. For example, Denmark has long had one of the world's highest tax rates on new cars, but with little historical variation that could be used by reduced form approaches to predict how reductions in the tax would affect the Danish economy. Gillingham et al. (2022) structurally estimated an equilibrium model of ownership and trading of cars to recover consumer preferences under the high tax *status quo* regime, and used it to compute counterfactual equilibria under a variety of alterative tax rates. They showed that by reducing the tax on new cars and increasing the gas tax, Denmark can: 1) improve the welfare of most of its citizens, 2) increase total tax revenue from cars, and 3) reduce total CO_2 pollution. Credible structural models are a faster, cheaper way of predicting the effects of policy changes by "crash testing" computer simulations of new policies before actually implementing them.

IRL (Ng and Russell, 2000) builds on *reinforcement learning* (RL) (Sutton and Barto, 1998). Its goal is to develop intelligent agents that can perform complex robotic and intellectual tasks without the need for explicit programming or an explicit reward function to direct behavior. The success of ML is due to *supervised learning* where algorithms are trained to predict outcomes using labeled data. However supervised learning is data-intensive and does not naturally handle sequential decision making where actions have long-term consequences. RL overcomes this by training agents to maximize a reward through trial-and-error, via repeated dynamic interactions with the environment. When the reward function is known, RL can approximate optimal behavior without labeled examples. Chess is an example with a known reward (1 for a win, -1 for a loss and 0 for a draw). RL has been able to learn superhuman performance via *self-play,* i.e. repeatedly playing the algorithm against itself resulting in performance that exceeds the best human and

[1] In economics "policy" refers to government interventions (tax rates, regulations, etc.), whereas in RL/MDP it refers to a decision rule mapping states to actions.

computerized chess players, see e.g. Silver et al. (2018).

However, many AI and robotics tasks lack clear reward function definitions. Christiano et al. (2017) consider whether "to use reinforcement learning to train a robot to clean a table or scramble an egg. It's not clear how to construct a suitable reward function, which will need to be a function of the robot's sensors." IRL solves this by learning a reward function from limited data on human or expert demonstrations. Using the inferred reward function, this enables further training more rapidly and cheaply via RL simulations resulting in intelligent behavior consistent with human preferences in situations outside of those in a limited demonstration data set.[2] For example, Google Maps trip data (Barnes et al., 2024) revealed travelers' route preferences reflecting trade-offs between traffic, distance, hills, safety, and scenery. They massively scaled IRL to estimate a route preference function (360M parameters, 110M trips) enabling RL to generate recommendations achieving 16-24% improvement in global route quality, and to the best of our knowledge, represents the largest published study or IRL algorithms in a real-world setting to date.

Section 2 reviews MDPs, the common ancestor of SE and RL. Section 3 covers DDC estimation and identification. Section 4 summarizes IRL. Section 5 discusses differences and open problems.

2 Background on DP and MDPs

SE and RL build on a recursive mathematical characterization of optimal sequential decision making under uncertainty known as *dynamic programming* (DP), Bellman (1957). We focus on stationary, infinite horizon Markovian Decision Problems (MDP), a subclass of DPs introduced by Bellman and Howard (1960) that most frequently used in SE and RL. MDPs are defined by 1) a state space S, 2) an action space A, and 3) the objects $\{r, p, \beta\}$, where $r(s, a)$ is the *reward or utility function* providing the payoff to a decision maker (DM), $p(s'|s, a)$ is a *transition density* for next period's state when the state is s and action is a and $\beta \in (0, 1)$ is a discount factor. Assume that S and A are finite sets with $|S|$ and $|A|$ elements, respectively. A policy or *decision rule* π is a conditional probability of choosing a feasible action $a \in A(s)$ for all $s \in S$. It is called a *mixed strategy* in game theory to distinguish it from a *pure strategy* where π puts probability 1 on a single action $a = \delta(s) \in A(s)$. The DM's objective is to choose a policy π to maximize the expected discounted sum of rewards given by

$$V(s) = \max_{\pi} V_{\pi}(s), \quad \text{where } V_{\pi}(s) = E_{\pi}\left\{\sum_{t=0}^{\infty} \beta^t r(s_t, a_t)|s_0 = s\right\}, \tag{1}$$

where E_{π} denotes the expectation operator over the Markov process $\{s_t, a_t\}$ implied by the transition probability p and policy π. The optimal value function can be defined recursively (Bellman,

[2] IRL is closely related to Reinforcement Learning from Human Feedback (RLHF), which trains agents using RL where it is unclear how to specify a reward function, instead learning from human preferences (Christiano et al., 2017).

1957) via *Bellman's equation*

$$V(s) = \max_{a \in A(s)} \left[r(s,a) + \beta \sum_{s' \in S} V(s') p(s'|s,a) \right] \equiv \Gamma(V)(s). \tag{2}$$

Thus, V is the fixed point $V = \Gamma(V)$ to an operator $\Gamma(V)$ known as the "Bellman operator" which can be shown to be a *contraction mapping* (i.e. for any vectors $V, W \in R^{|S|}$ we have $\|\Gamma(V) - \Gamma(W)\| \leq \beta \|V - W\|$ where β is less than 1 and $\| \cdot \|$ is the Euclidean norm). This implies that a unique solution V to Bellman's equation exists, an an optimal decision rule δ^* can be recovered from V by

$$\delta^*(s) = \underset{a \in A(s)}{argmax} \left[r(s,a) + \beta \sum_{s' \in S} V(s') p(s'|s,a) \right]. \tag{3}$$

Note that equation (3) implies that the optimal policy is generically a pure strategy, i.e. except for ties, state s has a unique $a^* \in A(s)$ that maximizes the right hand side of (3), implying a degenerate policy $\pi^*(a|s) = \mathbf{1}\{a = \delta^*(s)\}$. We now summarize the key algorithms for solving Bellman's equation for V and δ^*.

2.1 Successive Approximations

Since a contraction mapping Γ has a unique fixed point $V = \Gamma(V)$, the contraction property guarantees that the sequence $\{V^{(k)}\}$ given by $V^{(k)} = \Gamma(V^{(k-1)})$, for $k = 1, 2, \ldots$ starting from any initial starting guess $V^{(0)}$ converges to the fixed point V. This method, called *successive approximations* (SA), converges to V at a geometric rate in β but is very slow when β is close to 1.

2.2 Policy Iteration

The Howard (1960) *policy iteration* (PI) algorithm is a faster method for solving MDPs that alternates between *policy valuation* and *policy improvement* steps. At iteration k the PI algorithm has a candidate deterministic decision rule $\delta^{(k)}(s)$. The policy valuation step calculates the value of δ^k, $V_{\delta^{(k)}}$ by solving the linear system

$$V_{\delta^{(k)}} = r_{\delta^{(k)}} + \beta E_{\delta^{(k)}} V_{\delta^{(k)}}, \tag{4}$$

where $r_{\delta^{(k)}}$ is the vector whose s^{th} element is $r(s, \delta^{(k)}(s))$ and $E_{\delta^{(k)}}$ is the $|S| \times |S|$ matrix whose element in row s and column s' is given by $E_{\delta^{(k)}}(s,s') = p(s'|s, \delta^{(k)}(s))$. For any feasible deterministic rule δ the matrix E_δ is a Markov transition probability matrix with matrix norm $\|E_\delta\| = 1$, which implies that equation (4) has a unique solution given by $V_{\delta^{(k)}} = [I - \beta E_{\delta^{(k)}}]^{-1} r_{\delta^{(k)}}$. Given the value $V_{\delta^{(k)}}$ the policy improvement step calculates a new decision rule $\delta^{(k+1)}$ given by the right hand side of equation (3) except with $V_{\delta^{(k)}}$ substituted for V. If $\delta^{(k+1)} = \delta^{(k)}$ then policy iteration

has converged and it is not hard to show that $V_{\delta^{(k)}} = V$, the unique solution to Bellman's equation (2).

2.3 Randomization, function approximation, and the Curse of Dimensionality

SE and IRL repeatedly solve MDPs to find a reward function whose implied optimal behavior best approximates observed behavior according to different metrics of goodness of fit. It is already challenging to solve a single MDP when $|S|$ or $|A|$ is large since each SA or PI step requires $O(|A||S|^2)$ arithmetic operations to compute the expectation and maximization operators in equation (2) for all $s \in S$. Though PI requires only a few iterations to converge and thus can be faster for smaller problems when β is near 1, the downside is the $O(|S|^3)$ operations required to solve the linear system (4) in each policy valuation step. In the absence of sparsity or other "special structure" alternative methods are required when $|S|$ or $|A|$ is large.

Bellman called this the *curse of dimensionality* because it arises when discretization and interpolation are used to approximate solutions to MDPs with mulitple continuous state variables. Approximating a continuous MDP with d state variables using a finite grid of N points in each dimension results in an approximate finite MDP with $|S| = O(N^d)$ states which grow exponentially in d. The curse of dimensionality is unavoidable (Chow and Tsitsiklis, 1989): MDPs with d_s continuous state and d_c continuous choice variables have a worst-case complexity (minimum number of arithmetic operations to compute an ε-approximation to V) that is bounded below by a function that grows at rate $1/[(1-\beta)\varepsilon)]^{2d_s+d_c}$, a lower bound that holds for *any possible algorithm.*

It may be possible to break the curse of dimensionality for subclasses of MDPs with special structure, such as DDCs where the choice set A is finite. When $|A|$ is small, most of the work to solve the MDP is spent on 1) numerical integration to compute conditional expectations $EV(s,a) = \int V(s')p(s'|s,a)$, and 2) interpolation/approximation needed to find $V(s')$ for $s' \in S$. While randomization techniques such as Monte Carlo integration have been proposed to mitigate the curse of dimensionality (Rust, 1997), their general applicability remains debated (Bray, 2022).

Another way to break the curse of dimensionality (Bellman and Dreyfus, 1959) is using a parametric function V_ϕ that depends on a small number of unknown parameters ϕ chosen to make V_ϕ approximately solve the Bellman equation. For example V_ϕ could be the linear combination $V_\phi(s) = \sum_{k=1}^K \phi_k \rho_k(s)$ where $\{\rho_k\}$ are "basis functions" and $\phi = (\phi_1, \ldots, \phi_K)$ are coefficients, or V_ϕ could be a nonlinear function of ϕ such as a neural network. Neural networks are universal approximators, which might help break the curse of dimensionality of approximating V.

Let $V_{\hat{\phi}}$ denote a best approximation to V, where $\hat{\phi}$ minimizes the sum of squared "Bellman residuals" over a grid of points $(s_1, \ldots, s_J)$ in S

$$\hat{\phi} = \underset{\phi}{argmin} \sum_{j=1}^{J} [V_\phi(s_j) - \Gamma(V_\phi)(s_j)]^2. \tag{5}$$

Neural net approximation breaks the curse of dimensionality by allowing us to approximate V with

a small number of network weights/parameters ϕ. Randomization also helps break the curse of dimensionality when we use Monte Carlo integration instead deterministic quadrature to compute the conditional expectation entering the Bellman updates, $\Gamma(V_\phi)(s_j)$, and avoid numerical integration to calculate the mean squared Bellman error over the state space S in (5). This transforms the solution of the Bellman equation for V into a type of nonlinear regression problem.

Unfortunately, even if ϕ has relatively few elements and we can evaluate the Bellman errors in (5) rapidly, there may still be a curse of dimensionality associated with finding a global mimimizer $\hat{\phi}$.[3] Further, a $V_{\hat{\phi}}$ that approximately minimizes the sum of squared Bellman errors (5) need not be close to the true V that solves Bellman's equation (2). For example, if we start with the true solution $V = \Gamma(V)$ and perturb it by adding a constant C to V, the perturbed solution has a Bellman error of $(1-\beta)C$ which can be small even for large C when β is close to 1. Fortunately, DDCs do not require precise approximation of V to accurately approximate the optimal policy in (3) since shifting V by a constant leaves δ^* unaffected.[4]

2.4 Reinforcement Learning

This is a class of stochastic iterative algorithms for approximating the value function V and the optimal policy π inspired by models of trial and error learning by animals and humans. One of the first RL methods was *temporal difference learning* (TD) Sutton (1988) which can be regarded as a type of *stochastic approximation* (Robbins and Munro, 1951) for solving the linear system (4) for V_δ in the policy valuation step of PI. TD uses simulated or realized outcomes from $p(s'|s,\delta(s))$ in an iterative updating process for V_δ rather than numerical integration to compute the expectation operator E_δ in (4) and finding V_δ in a single step by solving a linear system with $|S|$ equations and unknowns. Instead TD requires many light iterations t that result in a stochastic sequence $\{\tilde{V}_\delta^t\}$ which is updated only at the realized state s_t using the random successor state $\tilde{s}_{t+1} \sim p(s'|s_t,\delta(s_t))$ via the recursion

$$\tilde{V}_\delta^{t+1}(s_t) = \tilde{V}_\delta^t(s_t) + \alpha_t \left[r(s_t,\delta(s_t)) + \beta \tilde{V}_\delta^t(\tilde{s}_{t+1}) - \tilde{V}_\delta^t(s_t) \right], \tag{6}$$

where $\alpha_t \geq 0$ is a step-size or learning rate sequence that must tend to zero, though not too fast. Under certain conditions on $\{\alpha_t\}$ the sequence of values $\{\tilde{V}_\delta^t\}$ from (6) converges with probability 1 to the vector V_δ that solves the system $V_\delta = r_\delta + \beta E_\delta V_\delta$.

Another example is *Q-learning* (Watkins, 1989), originally formulated as "a general formal model of an animal's behavioural choices in its environment." The Q function is the *choice-specific*

[3] Though stochastic optimization methods such as stochastic gradient descent can be helpful in avoiding getting stuck at local minima, randomization does not break the curse of dimensionality of continuous non-convex optimization (Yudin and Nemirovsky, 1976).

[4] Fujimoto et al. (2022) show that Bellman error is a poor proxy for value error in offline RL. Nguyen (2025) shows that in DDC problems, we obtain more accurate solutions for δ^* by minimizing the sum of squared *anchored* Bellman errors, $V_\phi(s_j) - V_\phi(s_1) - [\Gamma(V_\phi)(s_j) - \Gamma(V_\phi)(s_1)]$ that difference out parallel shifts in V_ϕ, allowing the minimizer to more accurately approximate the *shape* of V by ignoring its level.

value function implied by the Bellman equation

$$Q(s,a) = r(s,a) + \beta \sum_{s'} V(s')p(s'|s,a), \tag{7}$$

$$V(s) = \max_{a \in A(s)} Q(s,a). \tag{8}$$

Watkins proposed a stochastic iterative algorithm for updating Q given by

$$Q_{t+1}(s_t,a_t) = Q_t(s_t,a_t) + \alpha_t \left[r(s_t,a_t) + \beta \max_{a \in A(s_t)} [Q_t(\tilde{s}_{t+1},a)] - Q_t(s_t,a_t) \right], \tag{9}$$

where (s_t, a_t) is the state and action taken by the DM at time step t, and $\tilde{s}_{t+1}$ is a random draw from the transition probability $p(s'|s_t,a_t)$. Q-learning is also a form of stochastic approximation and Tsitsiklis (1994) showed that if the step size sequence $\{\alpha_t\}$ converges to 0 at a sufficiently slow rate, the sequence $\{Q_t\}$ converges with probability 1 to the unique Q that solves (7) and (8).[5] Equation (9) does not specify how actions a_t is chosen: a typical choice is the *greedy policy* $a_t = argmax_{a \in A(s_t)} Q_t(s_t,a)$. A key condition for the convergence of Q-learning is that it samples each pair (s,a) infinitely often. This implies the usual exploration/exploitation tradeoff since suboptimal actions must be taken infinitely often for order for Q learning to converge, which need not happen under a greedy policy. One solution is to use a *ε-greedy policy* that takes the optimal action a_t with probability $1-\varepsilon$, and with probability ε a randomly selected suboptimal action (i.e. an action a with $Q_t(s_t,a) < Q_t(s_t,a_t)$). However unless ε converges to 0 with t, Q_t will converge to a Q that does not satisfy the Bellman equation (8) since an ε-greedy policy is suboptimal.

Soft-Q learning (Haarnoja et al., 2017), building on *maximum entropy reinforcement learning* (Ziebart et al., 2008), provides an alternative way to generate continuous exploration, ensuring that all actions are explored infinitely often. These result in policies that converge to mixed strategies that are self-consistent in the sense that the corresponding Q satisfy a modified version of the Bellman equation (2) they refer to as the *soft Bellman equation.* The term "maximum entropy" refers to the inclusion of an "entropy regularization" term in the agent's objective function to result in optimal policies that are generically mixed. They modified the objective in equation (1) to include an entropy term, $H(\pi(s)) = -\sum_{a \in A(s)} \pi(a|s) \log(\pi(a|s))$, resulting in a dynamic optimization given by

$$V(s) = \max_{\pi} V_\pi(s), \quad \text{where } V_\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \beta^t [r(s_t,a_t) + \sigma H(\pi(s_t))] | s_0 = s \right\}, \tag{10}$$

where σ is the weight on entropy. Note that when $\sigma = 0$ we obtain the original MDP objective (1) and the optimal π is generically pure and has 0 entropy, but as $\sigma \to \infty$ the agent's optimal policy is simply to choose each feasible action with equal probability, the maximum entropy solution. The

[5] The conditions on the step size are 1) $\sum_t \alpha_t = \infty$ and 2) $\sum_t \alpha_t^2 < \infty$. Note $\alpha_t = 1/t$ satisfies these conditions.

value V to the modified DP problem (10) satisfies a "soft Bellman equation" which turns out to be identical to a version of the "smoothed Bellman equation" derived by Rust (1988), see equations (17) and (16) of section 3.

We call a method *model-free* if it does not require explicit specification or estimation of $p(s'|s,a)$; *model-based* otherwise. For inverse problems (IRL), model-free additionally means no parametric specification of $r(s,a)$ is required.

Q-learning is described as *model-free* since, unlike SA or PI, it does not require a specification for the transition probability $p(s'|s,a)$ or the need for numerical integration: we only need to *simulate new states using* p. In Watkins' example of an animal learning to adapt and thrive, p is implicit in its environment which "draws" the successor states s_{t+1} needed to implement the update in Q_t in equation (9). When trained in this manner Q-learning is indeed "model-free" and so are other RL algorithms that interact directly with the environment.

In many situations Q-learning and related RL methods produce policies that are too noisy and converge too slowly to be useful in real applications, especially when the number of states is large. As Jiang and Xie (2024) note, their "decisions can lead to undesirable outcomes, especially at the early stages of learning when the algorithm has little knowledge of the environment. This is not a problem when the environment is a simulator, but can lead to serious consequences when people are part of the environment." As a result RL training is increasingly done in simulation environments with sufficient computer power to rapidly conduct many thousands or millions of training iterations to produce a much better solution than could be achieved from limited real-world experience. However when RL is trained using computer simulations: if the simulator uses an explicit model of $p(s'|s,a)$ it is model-based; if it uses historical transitions (s,a,s') for TD updates, it is model-free.[6] The ability to simulate from the true $p(s'|s,a)$ (or a good approximation to it) for a large number of iterations is key to producing good solutions using RL.

Tabular (i.e. finite state/action) Q-learning, (9), is far too slow for MDPs with large state or action spaces. A scalable solution to this problem to use a Deep Q-Network (DQN), which is a deep convolutional neural network with parameters ϕ that parameterizes the Q-function as $Q_\phi(s,a)$ similar to the way V is approximated by V_ϕ in (5). This transforms the problem from iteratively updating Q-values, as in (9) to using stochastic gradient descent to find ϕ that minimizes the mean squared Bellman error $\rho(\phi)$ corresponding to (8) given by

$$\rho(\phi) = \sum_{i=1}^{N} \left[r(s_i,a_i) + \beta \sum_{s'} \max_{a \in A(s_i)} Q_\phi(s',a) p(s'|s_i,a_i) - Q_\phi(s_i,a_i) \right]^2, \tag{11}$$

over a grid of N points $\{(s_i,a_i)\}$. However the problem with directly minimizing $\rho(\phi)$ over ϕ in (11)

[6] The use of Q-learning to train chess strategies is an interesting intermediate case. Although the functional form of $p(s'|s,a)$ for chess is unknown since this transition probability depends on the action of the opponent, Q-learning remains "model-free" since it relies on the environment to produce the next state s_{t+1} directly via self-play, without explicitly modeling or using the game's rules to calculate $p(s'|s,a)$. In contrast, an algorithm like AlphaZero that uses Monte Carlo tree search explicitly uses the game rules for planning and would be considered "model-based".

is that forming a grid is challenging and the transition probability p is typically unknown. Also, as already noted in section 2.3, minimizing Bellman error does not ensure recovery of the optimal value function. For example, suboptimal policies can achieve low Bellman error due to overfitting (Fu et al., 2019).

A stochastic iterative procedure (Mnih et al., 2015) converges to a $\hat{\phi}$ that approximately minimizes the Bellman error $\rho(\phi)$; it was applied to train effective strategies for classic Atari 2600 video games. Similar methods were subsequently used for board games such as chess, Go and Shogu, see Silver et al. (2018). Simulated data were generated using *self-play* where each player makes choices using a trial policy π resulting in a simulated data set $\mathcal{D}$ of (s, a, s') sequences. Then they used stochastic gradient descent to minimize an approximation to the squared Bellman error (11) using the mean squared *temporal difference error*

$$\rho(\phi) = \mathbb{E}_{(s,a,s')\sim\mathcal{D}}\left[\left(r(s,a) + \beta \max_{a'} Q_{\hat{\phi}}(s',a') - Q_{\phi}(s,a)\right)^2\right], \tag{12}$$

where $\mathbb{E}_{(s,a,s')\sim\mathcal{D}}$ denotes the sample average over the simulated data set $\mathcal{D}$. Note that in (12) $\hat{\phi}$ is a lagged value of the network parameters from previous training iterations. Use of $\hat{\phi}$ instead of ϕ in (12) is motivated by the fact that the combination of off-policy learning, bootstrapping from subsequent estimates, and the use of a flexible nonlinear function approximator (the 'deadly triad') can lead to instability and non-convergence as noted by Sutton and Barto (1998).

Two innovations in the DQN avoided these problems. The first was *experience replay*, whereby transitions are stored in a large "replay buffer" $\mathcal{D}$. The network is then trained on mini-batches of transitions sampled uniformly randomly from $\mathcal{D}$, breaking the strong temporal correlations in the sequence of observations (e.g. persistent streaks in certain parts of the state space), thereby stabilizing the updates by conforming more closely to the i.i.d. assumptions underlying most stochastic gradient methods. The second innovation is the use of a separate *target network* with parameters $\hat{\phi}$, so that $Q_{\hat{\phi}}$ is a lagged version of the current network Q_{ϕ}. Keeping the target network's parameters $\hat{\phi}$ fixed for several iterations of the overall stochastic descent algorithm, keeps the optimization target stable, preventing the current network from attempting to converge to a non-stationary objective. Together, these innovations mitigated the deadly triad and enabled Q-learning to scale to high-dimensional problems that were previously intractable.[7]

[7] A single deep RL agent learned to play a diverse suite of Atari 2600 games (Mnih et al., 2015) directly from raw pixel inputs and game scores. It reused the same neural network architecture and hyperparameters across all tasks. Without any game-specific engineering, the DQN agent achieved human-level performance in several games. Despite this achievement, it should be highlighted that they did not demonstrate that their DQN converged to the optimal value function as the convergence of the approach is still an open question.

3 Structural Estimation of DDCs

The term "structural" refers to the attempt to infer the underlying preferences and beliefs of economic agents rather than simply summarizing their behavior, the main goal of reduced-form econometrics. In the MDP framework, an agent's decision rule $\pi(a|s)$ is the "reduced-form model" capturing behavior, whereas structural econometrics infers the underlying preferences and beliefs $\{\beta, r, p\}$. Key references include Rust (1987), Rust (1988), Rust (1994), Aguirregabiria and Mira (2002), and the survey by Aguirregabiria and Mira (2010).[8]

An obstacle to inference is the *statistical degeneracy* of the optimal decision rule $\delta^*(s)$ in equation (3), i.e. the fact that it is generally a pure strategy (i.e. deterministic function of s). Different individuals in the same observed state s will typically make different choices, so to enable the model to explain this heterogeneity we augment the state variable as $x = (s, \varepsilon)$ where ε is a vector of idiosyncratic preference shocks observed by the agent but not by others. Even though $\delta^*(x, \varepsilon)$ is still a pure strategy from the agent's standpoint, it produces choices that appear random from the observer's standpoint. Under additional assumptions that include the assumption $A(s)$ is finite for all $s \in S$, we can derive a *conditional choice probability* (CCP) $\pi(a|s)$ implied by the strategy δ^* that admits positive probabilities for all feasible choices $a \in A(s)$. Using this, we can form a likelihood function to infer $\{\beta, r, p\}$ from data on agents' states and choices.

Building on McFadden (1973)'s static discrete choice models, we can derive a simple expression for the CCP $\pi(a|s)$ known as the *multinomial logit model* or "softmax" function in the RL literature. Suppose that for each s, ε is a vector with the same length as $|A(s)|$ the number of actions, so for $\varepsilon(a)$ is the component of ε associated with action $a \in A(s)$. Assume that the reward for action a in state (s, ε) can be written as $r(s, a) + \varepsilon(a)$. We also assume that the transition density $p(s', \varepsilon'|s, \varepsilon, a)$ factors as $g(\varepsilon'|s')p(s'|s, a)$, for a conditional density $g(\varepsilon'|s')$. If we assume ε has a multivariate Gumbel or Type 1 extreme value distribution given by

$$F(\varepsilon|s) = \prod_{a \in A(s)} \exp\{-\exp\{-(\varepsilon(a) - \mu)/\sigma\}\}, \tag{13}$$

where we set $\mu = -\sigma\gamma$ where γ is Euler's constant, then $E\{\varepsilon(a)|s\} = 0$ for $a \in A(s)$. Let $V(s, \varepsilon)$ be the optimal value of this MDP, the solution to

$$V(s, \varepsilon) = \max_{\pi} V_\pi(s, \varepsilon), \quad \text{where } V_\pi(s, \varepsilon) = E_\pi \left\{ \sum_{t=0}^{\infty} \beta^t [r(s_t, a_t) + \varepsilon_t(a_t)] | s_0 = s, \varepsilon_0 = \varepsilon \right\}. \tag{14}$$

[8] See Rust (2014) for a discussion of the origins of SE, dating back to work at the Cowles Foundation in the 1940s and the rationale for this literature including the argument of Robert E. Lucas (1976) that structural models are superior to reduced-form for counterfactual prediction of policy changes. The development of DP in the 1950s and 60s revolutionized economics by enabling models of sequential decision making under uncertainty, dynamic strategic interactions, and dynamic equilibria in markets which are key features of nearly all economic problems. DP lead to new dynamic economic theories based on *rational expectations* that showed how beliefs about uncertain future events affect the current choices of rational forward looking agents. This in turn lead to empirical methods for estimating and testing how well these new theories explain behavior, an area now known as *dynamic structural estimation* (DSE).

Under these assumptions we can write the Bellman equation (2) for $V(s,\varepsilon)$ and show it has the representation

$$V(s,\varepsilon) = \max_{a\in A(s)} [Q(s,a) + \epsilon(a)], \tag{15}$$

where $Q = \Lambda_\sigma(Q)$ is the unique fixed point to the "smoothed Bellman operator" Λ_σ given by

$$\Lambda_\sigma(Q)(s,a) = r(s,a) + \beta \sum_{s'} \sigma \log \left(\sum_{a'\in A(s')} \exp\{Q(s',a')/\sigma\} \right) p(s'|s,a). \tag{16}$$

The derivation of (15) and (16) use an important property of the extreme value distribution for ε, namely that the expectation of $V(s,\varepsilon)$ with respect to ε has an analytical expression given by the so-called log-sum or "smoothed max function" $V(s)$

$$\begin{aligned} V(s) &\equiv E\left\{V(\tilde{s},\tilde{\varepsilon})\,|\tilde{s}=s\right\} = \int_\varepsilon \max_{a\in A(s)} \left[Q(s,a)+\varepsilon(a)\right] F(d\varepsilon|s) \\ &= \sigma \log \left(\sum_{a\in A(s)} \exp\{Q(s,a)/\sigma\} \right). \end{aligned} \tag{17}$$

Using equations (16) and (17) we can derive a "smoothed Bellman equation" for V

$$V(s) = \sigma \log \left(\sum_{a\in A(s)} \exp\left\{ [r(s,a) + \beta \sum_{s'} V(s')p(s'|s,a)]/\sigma \right\} \right). \tag{18}$$

Equation (16) defines a "smoothed Q" as the fixed point of the smoothed contraction mapping $Q_\sigma = \Lambda_\sigma(Q_\sigma)$, and (18) defines V as a fixed point of the smoothed Bellman operator (also a contraction mapping) $V_\sigma = \Gamma_\sigma(V_\sigma)$. These operators are equivalent to the corresponding operators for MDP without the state variable ε in equations (7) and (8) and it is not hard to show that Q_σ and V_σ converge to Q and V as $\sigma \downarrow 0$.

The representation of $V(x,\varepsilon)$ in equation (15) means that the decision rule $\pi(a|x,\varepsilon)$ is isomorphic to a static discrete choice model where $Q(s,a)+\varepsilon(a)$ is the utility/payoff from action a. The CCP $\pi(a|s)$ is the conditional probability that the DM chooses action $a \in A(s)$ given the observed state s, taking the multinomial logit form (McFadden, 1973)

$$\begin{aligned} \pi(a|s) &= \int I\left\{ Q(s,a)+\varepsilon(a) \geq \max_{a'\in A(s)} [Q(s,a')+\varepsilon(a')] \right\} F(d\varepsilon|s) \\ &= \frac{\exp\{Q(s,a)/\sigma\}}{\sum_{a'\in A(s)} \exp\{Q(s,a')/\sigma\}}. \end{aligned} \tag{19}$$

As $\sigma \downarrow 0$, $\pi(a|s)$ converges to a degenerate probability that equals 1 if $Q(s,a)$ is the largest Q value and 0 otherwise, i.e. to the optimal pure strategy π in equation (3) for MDPs without the ε variable. If follows that the choice-specific values $Q(s,a)$ in (16) and CCPs $\pi(a|s)$ in (19) are

exactly the same as the "soft Q" values and optimal policy derived in the RL literature under the assumption that agents care about entropy of the policy π, given in equation (10) of section 2.4.

Equation (19) can be considered as the analog of the *policy improvement* step for MDPs in equation (3). The analog of the *policy valuation* step (4) (Aguirregabiria and Mira, 2002) is the value function V_π while is the solution to the linear system

$$V_\pi(s) = \sum_{a \in A(s)} \pi(a|s)[r(s,a) + E\{\varepsilon(a)|s,a\} + \beta \sum_{s'} V_\pi(s')p(s'|s,a)], \tag{20}$$

where $E\{\varepsilon(a)|s,a\}$ is the conditional expectation of $\varepsilon(a)$ given that action a is chosen in state s under decision rule π. The extreme value assumption implies that $E\{\varepsilon(a)|s,a\} = -\sigma \log(\pi(a|s))$, and using equation (19), we see that right side of (20) equals the smoothed max function defining the smoothed Bellman operator in equation (18). It follows that we can use policy iteration to solve the smoothed Bellman equation $V_\sigma = \Gamma_\sigma(V_\sigma)$ cycling between policy valuation steps (20) and policy improvement steps (19) using the Q_π implied by V_π in equation (16).

Equation (19) implies that $\sigma \log(\pi(a|s)) = Q(s,a) - V(s) \equiv \mathcal{A}(s,a)$, where $\mathcal{A}(s,a)$ is called the *advantage function* in RL. Using $\mathcal{A}(s,a)$ we can rewrite the nonlinear fixed point equation (16) for Q as a linear system depending on $r(s,a)$ and $\log(\pi(a|s))$,

$$Q(s,a) = r(s,a) + \beta \sum_{s'} \sum_{a' \in A(s')} \left[-\sigma \log(\pi(a'|s')) + Q(s',a')\right] \pi(a'|s')p(s'|s,a). \tag{21}$$

Equation (21) implies that we can decompose Q as $Q(s,a) = Q_r(s,a) + Q_\varepsilon(s,a)$ where $Q_r(s,a)$ is the component of discounted value from observed rewards r and $Q_\varepsilon(s,a)$ is the component from unobserved rewards ε. Further substituting expression (20) in place of (18) in the equation for Q in (16), we can show that Q_r and Q_ε are each the solution to the following linear systems

$$Q_r(s,a) = r(s,a) + \beta \sum_{s'} \sum_{a' \in A(s')} Q_r(s',a')\pi(a'|s')p(s'|s,a) \tag{22}$$

$$Q_\varepsilon(s,a) = \beta \sum_{s'} \sum_{a' \in A(s')} \left[-\sigma \log(\pi(a'|s')) + Q_\varepsilon(s',a')\right] \pi(a'|s')p(s'|s,a). \tag{23}$$

This decomposition reduces the computational burden of estimating DDC models.

3.1 Estimation Methods for DDCs

The most common estimation method for DDCs is some form of maximum likelihood. Suppose we observe a random sample of *trajectories* i.e. data on the consecutive states and actions taken by N individuals $\{(s_{it}, a_{it})|t = 0, \ldots, T_i,\ i = 1, \ldots, N\}$. Suppose that we smoothly parametrize (r,p) using a vector unknown parameters θ, which we denote by r_θ and p_θ.[9] Using the implicit function

[9]The discount factor β is often treated as known, though in some cases it is also estimated and thus part of θ.

theorem, we can show that V_θ and Q_θ, the contraction fixed points given in equations (17) and (16), are continuously differentiable function of θ. Using these fixed points and the logit formula for $\pi(a|s)$ in equation (19) we can write a full likelihood function for the data as a function of the unknown parameters θ given by

$$L(\theta) = \prod_{i=1}^{N} \prod_{t=1}^{T_i} \pi_\theta(a_{it}|s_{it}) p_\theta(s_{it}|s_{it-1}, a_{it-1}). \tag{24}$$

The smoothness of V_θ and Q_θ in θ implies that the CCP π_θ and the likelihood $L(\theta)$ are both smooth functions of θ so the standard asymptotic efficiency properties of maximum likelihood apply.

The Nested Fixed Point algorithm (NFXP) (Rust, 1988) computes the MLE of θ via a standard outer hill-climbing loop that maximizes $L(\theta)$, where each evaluation calculates the contraction fixed point $Q_\theta = \Lambda_\theta(Q_\theta)$ that defines the choice-specific values Q_θ where Λ_θ is the operator defined on the right hand side of equation (16). Notice that $L(\theta)$ depends on θ via Q_θ. An alternative strategy (Su and Judd, 2012) computes the MLE via the MPEC algorithm, maximizing $L(\theta, Q)$ as a function of (θ, Q) subject to the fixed point constraint $Q_\theta = \Lambda_\theta(Q_\theta)$.[10] NFXP's computational burden stems from computing the inner fixed point $Q_\theta = \Lambda_\theta(Q_\theta)$ for each trial value of θ in the outer hill-climbing algorithm. Newton's method (e.g. policy iteration) is typically used to solve for Q_θ (or alternatively for V_θ in equation (20)). Even though Newton's method converges rapidly, it requires $O(|S|^3)$ operations per Newton/policy iteration step, making NFXP burdensome for large state spaces S. Therefore alternative estimation methods have been proposed that reduce the total number of policy iteration steps required to estimate θ.

The "CCP estimator" (Hotz and Miller, 1993) for r uses a non-parametric estimate of $\pi(a|s)$ to avoid repeated solution of Q over the search for θ that NFXP requires. Consider the case where r_θ is linear in parameters, i.e. we can write $r_\theta(s,a) = r(s,a) * \theta \equiv \sum_{k=1}^{K} r_k(s,a)\theta_k$ for known functions $\vec{r} = (r_1, \ldots, r_K)$ called *features* in the IRL literature. Then (22) implies that $Q_r(s,a) = R(s,a) * \theta$ where R is the solution to

$$R(s,a) = \vec{r}(s,a) + \beta \sum_{s'} \sum_{a' \in A(s')} R(s',a')\pi(a'|s')p(s'|s,a) = \vec{r}(s,a) + \beta ER(s,a), \tag{25}$$

where $ER(s,a)$ is a shorthand for the conditional expectation of $R(s,a)$ the middle of expression in (25). It follows that we only need to solve (25) once using a non-parametric estimate of $\hat{\pi}$ along with Q_ε in (23) to serve as a "correction term" to obtain the following form for the CCP

$$\pi_\theta(a|s) = \frac{\exp\{\hat{R}(s,a) \cdot \theta + \hat{Q}_\varepsilon(s,a)\}}{\sum_{a' \in A(s)} \exp\{\hat{R}(s,a') \cdot \theta + \hat{Q}_\varepsilon(s,a')\}}, \tag{26}$$

[10] Su and Judd (2012) claim that the MPEC alogithm is substantially faster than NFXP, but this was due to using SA (Iskhakov et al., 2016) to compute the fixed point $Q_\theta = \Lambda_\theta(Q_\theta)$, which is slow when β is close to 1. When Newton's method is used to compute this fixed point, the cpu times for MPEC and NFXP are roughly comparable with NFXP being faster on test problems with large sample sizes.

where $\hat{R}(s,a)$ is the solution to (25) and $\hat{Q}_\varepsilon$ is the solution to (23) using non-parametric first stage estimators $\hat{\pi}(a|s)$ and $\hat{p}(s'|s,a)$. The CCP estimator $\hat{\theta}$ maximizes the partial likelihood $L_\pi(\theta)$ given by

$$L_\pi(\theta) = \prod_{i=1}^{N}\prod_{t=1}^{T_i} \pi_\theta(a_{it}|s_{it}), \tag{27}$$

where π is given in (26).[11] Thus, when r is a linear combination of known features, we only need to compute $\hat{R}$ and $\hat{Q}_\varepsilon$ once at the start of the estimation, requiring $K+1$ solutions of systems with $|S|$ equations and unknowns, where K is the number of features. This can be further reduced to a single solution if we assume that $r(s,a_s)$ is known for some "normalizing alternative" $a_s \in A(s)$ (see Section 3.3).

A cost of the CCP estimator is that it is less efficient than using NFXP to estimate the same partial likelihood criterion (27) due to the noise in the first stage non-paramteric estimates $\hat{\pi}$ that are used to construct the $\hat{Q}_\varepsilon$ function in (26). Aguirregabiria and Mira (2002) (AM) addressed this problem by proposing an iterative estimator called *nested pseudo likelihood* (NPL) that uses an initial non-parametric estimate $\hat{\pi}$ to compute the value function $V_{\hat{\pi}}$ implied by this initial estimate by solving for it in the policy valuation step in equation (20), which we denote as $\hat{V}_{\theta,\hat{\pi}}$ since it depends both on $\hat{\pi}$ and the parameters θ entering the reward function. AM then estimate $\hat{\theta}$ using a pseudo-likelihood similar to (27) except that the CCP $\pi_\theta(a|s)$ is evaluated using (19) using estimated Q functions given by

$$\hat{Q}_\theta(s,a) = r_\theta(s,a) + \beta \sum_{s'} \hat{V}_{\theta,\hat{\pi}}(s'|s,a)p(s'|s,a), \tag{28}$$

Thus, the likelihood $L_\pi(\theta)$ as depends implicitly on the first stage non-parametric CCP estimate $\hat{\pi}$, which can be regarded as a "nuisance parameter". In principle, estimation noise in $\hat{\pi}$ will contaminate and affect the asymptotic covariance matrix for the parameters of interest θ. However AM established a key *zero Jacobian property* that at the fixed point $\partial V(s)/\partial\pi = 0$ which implies that the first stage nuisance parameter $\hat{\pi}$ is "Neyman orthogonal" with respect to the parameters of interest θ in the sense of Chernozhukov et al. (2022). It follows that the asymptotic covariance matrix for the NPL estimator of θ is unaffected by noise in the first stage $\hat{\pi}$ and is the same as π was known. Further, AM showed that NPL can be iterated: using $\hat{\theta}$, they can compute an updated estimate of the CCPs $\hat{\pi} = \pi_{\hat{\theta}}$) and repeat the policy valuation step to get a new set of Q functions (28), and use these in the likelihood (27) to get an updated estimate $\hat{\theta}$. They refer to NPL as "swapping" the policy iteration and maximization steps of the NFXP estimator (which does maximization in the "outside" loop and policy iteration in the "inside" or nested fixed point loop). Swapping the order of maximization and policy iteration reduces the total number of policy iteration steps required to estimate θ. AM showed that if this iterative version of NPL converges,

[11] See Chernozhukov et al. (2022) for a modified version of the CCP estimator that is "locally robust" i.e. it corrects for sampling error in the first stage estimates of the functions $\hat{H}(s,a)$ and $Q_\varepsilon(s,a)$.

it produces the same estimate $\hat{\theta}$ that NFXP produces from maximization of the partial likelihood (27). Thus the NPL estimator is as efficient as NFXP but at lower computational cost, and it is more efficient than the CCP estimator but at a higher computational cost.[12]

Bayesian approaches have been proposed for inference in DDC models using Markov Chain Monte Carlo methods that simultaneously simulate the likelihood as well as stochastic simulations to compute estimates of value functions that are similar to "Monte Carlo tree search" used by RL algorithms such as the Q-learning algorithm discussed in section 2.4, see Imai et al. (2009). There are also non-Bayesian simulation estimators such as Bajari et al. (2007) that estimate θ as the value that minimizes the degree of suboptimality in observed actions relative to other actions not taken using simulation estimates of Q function evaluated at the actions the DM actually took and other actions not chosen.

3.2 Function Approximation Methods for Large State Spaces

When the state space S is large (i.e. there are continuous state variables that must be discretized) the approaches discussed above that use PI and repeated solution of large linear systems with $|S|$ equations and unknowns become computationally infeasible. Following section 2.3 we can approximate the fixed point $Q = \Lambda_\sigma(Q)$ of the smoothed Bellman operator in equation (16) by $Q_\phi(s)$ with a flexible parametric family such as a neural network with weights ϕ, and solve the fixed point problems (16) and (17) more tractably as nonlinear least squares problems. Let $\rho(\theta, \phi)$ be the sum of squared residuals of the smoothed Bellman operator, $Q_\phi - \Lambda_\sigma(Q_\phi)$, over a grid of N points in the state space, equation (11). Note that the residuals also depend on θ via $\{\beta, r, p\}$ in the smoothed Bellman operator Λ_σ. A nested least squares estimator similar to NFXP can estimate θ by maximizing the likelihood $L(\theta)$ (24) while mimimizing $\rho(\theta, \phi)$ over ϕ for each value of θ. A penalized likelihood estimator called SEES (Luo and Sang, 2024), for *sieve-based efficient estimator for structural models*, chooses $\hat{\theta}$ and $\hat{\phi}$ to maximize

$$(\hat{\theta}, \hat{\phi}) = \underset{\theta,\phi}{argmax} \log(L(\theta)) - \lambda\rho(\theta, \phi), \tag{29}$$

where $\lambda \geq 0$ is a penalization parameter. Consistency of SEES requires the use of a *sieve,* i.e. the dimension of ϕ must grow to infinity at certain rate with the sample size so that the approximation error in approximating Q by Q_ϕ converges to zero. A related estimator called *neural network efficient estimator* (NNES) (Nguyen, 2025) maximizes a penalized criterion similar to the SEES estimator (29) but in a sequential fashion similar to the NPL estimator of AM.

A challenge facing sieve-based estimators is that ϕ are "nuisance parameters" (their only purpose is to approximate Q) whereas θ are the "parameters of interest" that determine rewards and beliefs, $\{\beta, r, p\}$. Since $\hat{\phi}$ minimizes the sum of squared residuals $\rho(\phi, \theta)$ for each θ, it follows the $\hat{\phi}$ will

[12] See also *efficient pseudo likelihood* (EPL) (Dearing and Blevins, 2025), a similar iterative procedure that is also an efficient sequential estimator that can estimate parameters of dynamic games.

be an implicit function of θ. The asymptotic covariance matrix for $\hat{\theta}$ requires $\nabla_\theta Q$, the gradient of Q with respect to θ. However directly differentiating $Q_{\hat{\phi}(\theta)}$ with respect to θ using the chain rule $\nabla_\theta Q = \nabla_\phi Q_{\hat{\phi}(\theta)} \nabla_\theta \hat{\phi}(\theta)$ yields inaccurate results due to the high dimensionality of ϕ and the fact that the minimization over ϕ can result in local optima and fragile solutions that can cause small changes in θ to produce large changes in $\hat{\phi}(\theta)$. Nguyen (2025) addresses this problem by differentiating both sides of the fixed point equation for Q (21) with respect to θ to obtain a separate fixed point equation that uniquely determines $\nabla_\theta Q$. He approximates this solution by parametrizing $\nabla_\theta Q$ by a separate neural net with weights ψ, and shows that the solution $\nabla_\theta Q_{\hat{\psi}}$ accurately approximates to $\nabla_\theta Q$. However doing this involves the computational burden of having to fit two neural nets each time θ is updated: one for $Q_{\hat{\phi}(\theta)}$ and the other for $\nabla_\theta Q_{\hat{\psi}(\theta)}$.

3.3 The Identification Problem

Section 2 showed how π is calculated from the structure $\{\beta, r, p\}$, which we summarize by the mapping $\pi = f(\beta, r, p)$. The Identification Problem asks if this mapping is invertible: given π is there a unique underlying structure $\{\beta, r, p\}$ that implies it? Unfortunately, without further restrictions the answer is negative, as shown by Hotz and Miller (1993), Rust (1994) and Magnac and Thesmar (2002). To understand why, consider the first step of the inversion process, uncovering Q from π. Using the logit form of π in (19) we have

$$\log\big(\pi(a|s)/\pi(a'|s)\big) = Q(s,a) - Q(s,a'), \quad s \in S,\ a, a' \in A. \tag{30}$$

We already see that information on π is insufficient to recover all the Q values: only the *differences* in Q are identified but not all $|S||A|$ possible values of $Q(s,a)$. Indeed, the structure $\{\beta, r, p\}$ contains $1 + |S|(|A|-1) + |S|^2|A|$ parameters which is more than then $|S|(|A|-1)$ parameters of π, so we will generally have "more equations than unknowns" and the underlying structure will only be partially identified.

Identification requires additional restrictions on $\{\beta, r, p\}$. Empirical studies commonly impose the assumption that r and p have known parametric functional forms that depend on a relatively small number of parameters θ. In some situations the reward function can be treated as known, and this facilitates the identification of subjective beliefs $p(s'|s,a)$.[13] In many applications researchers assume agents have rational expectations, i.e. their subjective beliefs about state transitions $p(s'|s,a)$ correspond to the actual probability governing these transitions. With sufficient data, p can be estimated non-parametrically and so for the analysis of identification it can be treated as known.[14] Many studies typically assume that the discount factor β is also known, how-

[13] For example Anderson et al. (2025) are able to identify the subjective beliefs of professional tennis servers about how their serve strategies affects their probability of winning a tennis game. These beliefs take the form of a transition $p(s'|s,a)$ where s denotes the point state of the game and a denotes serve direction. Identification of beliefs is possible because it is reasonable to assume r is known: i.e. the server earns a reward of 1 if they win the game and 0 otherwise, and due to reasonable parametric restrictions on p.

[14] With sufficient restrictions on r the rational expectations assumption is testable. For example, Anderson et al.

ever under the assumption of rational expectations and restrictions on rewards, β can be identified, see Abbring and Daljord (2020).

If $\{\beta, p\}$ are known, identification of r depends on the assumption that either it has a known parametric functional form r_θ where the dimension of θ is less than $|S|(|A|-1)$, or that for each $s \in S$ there is a normalizing or "anchor action" a_s such that $r(s, a_s)$ is known. The latter assumption amounts to $|S|$ restrictions that imply there are only $|S|(|A|-1)$ unknown rewards to be recovered, the same number of free values of π. Hotz and Miller (1993) and Kang et al. (2025) show that in this "exactly identified" case, we can calculate $Q(s, a_s)$ from knowledge of $r(s, a_s)$ and $\pi(a_s|s)$ using equation (21) and then (30) allows us to recover all remaining Q values. Using the Q values we can back out the remaining unknown rewards r from the smooth Bellman equation (16).

The assumption that we know $r(s, a_s)$ for some anchor action a_s in each state $s \in S$ is far from innocuous. If wrong, estimated rewards will not equal true rewards even though the model perfectly fits π, and counterfactual predictions may differ from actual outcomes. In the absence of prior restrictions on rewards, we can only identify "observationally equivalent" rewards of the form $r(s, a) + h(s) - \beta \sum_{s'} h(s')p(s'|s, a)$ for any function $h : S \to R$. This partially identified set defines an inherent limit on what can be learned about rewards without accurate prior knowledge of at least some aspects of the reward function.[15]

To resolve the identification ambiguity, the DDC and IRL literatures have developed distinct sets of identifying assumptions, each reflecting the typical problems and goals of the respective fields. Kalouptsidi et al. (2021) provide examples of certain counterfactuals that are identified even when r is only partially identified. They conclude that their "results call for caution while leaving room for optimism: although counterfactual behavior and welfare can be sensitive to identifying restrictions imposed on the model, there exists important classes of counterfactuals that are robust to such restrictions." (p. 385).

4 Inverse Reinforcement Learning

The IRL literature (Ng and Russell, 2000) has two motivations for inferring rewards: 1) "the potential use of reinforcement learning and related methods as potential models for human and animal learning" and 2) to provide a reward function that RL can use to construct an "intelligent agent that can behave successfully in a particular domain." Though they note that imitation and apprenticeship learning (IL and AL) can be used to learn a policy π, "the reward function often provides a much more parsimonious description of behaviour" so that IRL can constitute an effective form of AL. IRL has become an important method in many areas of robotics including

(2025) strongly reject the hypothesis the many professional tennis servers have rational beliefs about their own ability and those of their opponent, as embodied by $p(s'|s, a)$.

[15] This is what the IRL literature, following Ng et al. (1999), calls potential-based reward shaping, where $h(s)$ is a 'potential function' $\Phi(s)$. Any reward $r'(s, a) = r(s, a) + \beta\Phi(s') - \Phi(s)$ yields the same optimal policy: identification 'up to policy invariance.' For imitation learning, policy invariance suffices. But for structural econometrics, policy invariance is not enough.

development of autonomous drones and vehicles (AV) where there is insufficient data on decisions to human drivers under multitudes of different road conditions to develop effective AVs. Wang et al. (2025) note that IRL has "found great success in predicting trajectories through inferring cost or reward functions from expert demonstrations, and then using these functions to guide the behavior of self-driving vehicles in unseen driving environments."

We selectively review IRL methods overlapping with the DDC literature of Section 3. We refer readers to surveys by Arora and Doshi (2021), Gleave and Toyer (2022), and Adams et al. (2022) for a broader and more in depth view of IRL. The latter survey provides further examples of how IRL uses "an expert to demonstrate the successful task and then a highly parameterized reward function could be transferred to the robot for optimization" and how IRL can generate more effective policies than AL, particularly for new, counterfactual environments where "IRL has the added advantage that the reward function can be transferred to new environments with different dynamics.".

There are four broad classes of IRL algorithms: 1) max-margin, 2) Bayesian methods, 3) maximum entropy methods, and 4) adversarial methods. Three types of max-margin estimators (Ng and Russell, 2000) search for a reward function for which payoffs under the observed policy are higher than any other policy (i.e. behavior), an approach similar to the "moment inequality" simulation estimator (Bajari et al., 2007) discussed in section 3. Ng and Russell (2000) recognized the identification problem discussed in the previous section, namely there may be many reward functions that satisfy the max-margin inequalities (payoff inequalities) including the trivial solution $r = 0$.[16]

Bayesian IRL (BIRL) (Ramachandran and Amir, 2007) assumes π is a mixed strategy with the softmax form (19) that depends on Q functions which are implicit functions of r per the Bellman equations (7) and (8) and interpret the σ parameter in the softmax function for π as a rationality coefficient that governs how sharply the policy concentrates on high-value actions.[17] This setup allowed them to formulate a likelihood for the sequence of observed actions by the DM that depends on r. Using Bayes' Rule, they computed a posterior distribution for r, treating it as the unknown parameter with a prior distribution. Since it is infeasible to directly compute the posterior, BIRL uses MCMC to generate draws from the posterior distribution, with policy iteration used to solve for Q for each draw of r. This makes BIRL method computationally intensive for large state spaces similar to NFXP. Overall BIRL is similar to the Bayesian MCMC methods discussed in section 3, except that those methods use stochastic updates of Q similar to those produced by Q-learning to gradually compute Q over the course of the MCMC simulations rather than use policy iteration to find Q for each MCMC draw of r.

A seminal contribution to the IRL literature is Ziebart et al. (2008), who introduced *maximum entropy IRL* as a means of justifying the use of the softmax/logit probability for $\pi(a|s)$ as the probability with maximum entropy subject to certain constraints that make expectations of certain

[16] Ng and Russell (2000) termed this identification "up to policy invariance." See Section 3.3.

[17] Small σ makes the policy concentrate almost entirely on the highest-value action, while large σ spreads probability nearly uniformly across actions. Because the softmax formula depends only on Q/σ, the reward scale and σ cannot both be identified, so DDC applications typically fix $\sigma = 1$ (e.g. Adusumilli and Eckhardt (2025)). When calibration data are available, Ghosal et al. (2023) show how to estimate β (equivalently $\sigma = 1/\beta$) instead of fixing it.

functions known as *features* equal the sample averages of these features.[18] They proposed to estimate the parameters of a reward function that is a linear combination of a set of known features using a partial likelihood for observed (a, s) pairs that is identical to the partial likelihood (27) used in the DDC literature. They applied this method to model the route choices of 25 Yellow cab drivers in Pittsburgh with a reward function that involves four features: road type, speed, lanes and transitions. They showed that the estimated model could correctly predict nearly 80% of the cab paths in a holdout or evaluation sample and that their new method was a significant improvement over other IRL methods such as max-margin.

A fourth class of IRL methods uses adversarial training, building on Generative Adversarial Imitation Learning (GAIL). Fu et al. (2018) introduced Adversarial IRL (AIRL), which extends GAIL by learning a *disentangled* reward function that is robust to changes in environment dynamics. AIRL formulates IRL as a two-player game between a policy generator and a discriminator. The key innovation is structuring the discriminator as $f(s, a, s') = g_\theta(s, a) + \gamma h_\phi(s') - h_\phi(s)$, where $g_\theta(s, a)$ represents the learned reward function and h_ϕ is a state-dependent *shaping function*. This discriminator structure is a practical implementation of the potential-based reward shaping discussed in Section 3.3, where $h_\phi(s)$ plays the role of the potential function. As noted in our earlier discussion, this approach allows AIRL to achieve identification via the global 'reward-decomposability' assumption, but its foundational proof relies on deterministic dynamics, a key point of departure from the stochastic environments central to DDC modeling. This decomposition disentangles the reward from dynamics-specific features, enabling learned rewards to transfer across environments with different transition probabilities $p(s'|s, a)$. The discriminator is trained to distinguish expert trajectories from policy-generated trajectories, while the policy maximizes the learned reward. AIRL achieves identification of the reward function "up to policy invariance" (i.e., up to potential-based shaping) without requiring explicit solution of the Bellman equation during training.[19]

The adversarial framework has a direct analog in structural econometrics. Kaji et al. (2023) formalize this connection: a discriminator trained to distinguish real from simulated data automatically selects optimal moments. With a logistic discriminator this reduces to optimally-weighted simulated method of moments (SMM); with a neural network it approximates MLE without a closed-form likelihood. Lee et al. (2026) achieve identification not through post-hoc approximation, but by structurally constraining the adversarial game itself. By fixing the utility of a known "anchor action" (e.g., exit) to zero within the discriminator's objective, they mathematically eliminate the reward shaping ambiguity that typically plagues AIRL. This constraint allows them to

[18] Ziebart et al. (2010) extended these ideas to MDPs and called it *maximum causal entropy IRL*.

[19] However, Theorem C.1 in Fu et al. (2018) requires deterministic dynamics: "we also assume we have deterministic dynamics." Since economic applications typically involve stochastic transitions $p(s'|s, a)$, this assumption limits AIRL's applicability to most DDC problems. However, Lee et al. (2026) prove that action-dependent rewards are identified within AIRL when constrained by an exit anchor. Fu et al. (2018) demonstrate that AIRL achieves comparable performance to GAIL on standard imitation benchmarks but significantly outperforms GAIL on transfer tasks requiring generalization across different environment dynamics.

disentangle immediate utility from continuation values directly from high-dimensional text data, successfully recovering distinct latent segments—such as high-patience "Wait & Read" users versus high-willingness-to-pay "Pay & Read" users—solely from observed consumption trajectories.

The policy π maximizes the expected discounted entropy-regularized payoff in equation (10) (Haarnoja et al., 2017). Various criteria are used to estimate r such as "occupancy matching" (Ho and Ermon (2016), Yue et al. (2023)). For our purposes we focus on IRL where the likelihood function is the estimation criterion, such as Ermon et al. (2015), Zeng et al. (2023), Zeng et al. (2024), Barzegary and Yoganarasimhan (2022) and Kang et al. (2025).[20] This results in a nearly complete equivalence between the DDC and IRL literatures, with identical softmax/logit formulas for the policy $\pi(a|s)$ (19) that depend on the choice-specific or "soft Q" values that satisfy the "soft Bellman equation" in equation (16). Overall, DDC and maximum entropy IRL result in mathematically identical optimization problems for recovering r from data on states and actions of a sample of DMs. The main difference is in the algorithms used to solve the DM's DP problem.

In DDC the Q functions are computed by non-stochastic algorithms such as SA or PI, or in large state spaces by parametrizing Q or the value function V and finding parameters ϕ that minimize the squared "Bellman residuals" as in the SEES estimator, (29). All of these methods can be described as *model-based* because they require knowledge of the transition probability $p(s'|s,a)$, or at least that we can estimate p from data. Adams et al. (2022) notes that "Most IRL methods assume that a model (more specifically the transition function) is known to the learner and that it can be used to solve the forward RL problem to evaluate the quality of the estimated reward function. However, in many scenarios, the transition function may be unknown and difficult or impossible to estimate." (p. 4327). That is, RL requires the ability to at least simulate realizations from a known $p(s'|s,a)$ function to train decision rules. Model-free IRL avoids the need to parametrically specify or non-parametrically estimate p to use to simulate training outcomes. Instead a new class of nodel-free IRL methods avoid the need to construct or even simulate from p in a clever way: by using historical observations of (s', s, a) in data to take the place of a simulator needed by RL algorithms such as Q-learning. Sufficient data on past state transitions can be used with RL-inspired methods to estimate the value and Q functions and the reward function r, reminiscent of the idea of *experience replay* in the deep Q learning literature, Mnih et al. (2015). We discuss two recent IRL contributions that recover r from data in a model-free manner. One can be regarded as an IRL version of the CCP estimator and the other is similar to the SEES estimator of Luo and Sang (2024) or the NNES estimator of Nguyen (2025). Both methods use function approximation methods (expressing V and Q as outputs of neural networks or linear combinations of a set of basis functions) to handle problems with large state spaces.

Adusumilli and Eckhardt (2025) (AE) proposed a model-free estimator that combines a core method of RL, the method of *temporal differences* (TD), Sutton and Barto (2020), and the two step Hotz and Miller (1993) CCP estimator in section 3.1 that avoids the need to estimate or

[20] Zeng et al. (2023) and Zeng et al. (2024) maximize a discounted log-likelihood function.

make parametric assumptions about $p(s'|s,a)$. Similar to the CCP estimator, they assume the reward is a linear combination of known "features", i.e. $r_\theta(s,a) = \sum_{k=1}^K \theta_k r_k(s,a)$ and estimate the structural parameters θ by maximizing the partial likelihood (27). However rather than numerically computing functions R and Q_ϵ that enter as covariates in the CCP π in equation (26), they use observed state transitions to estimate R and Q_ϵ under the assumption they can be approximated as linear combinations of a set of known basis functions that depend on (s,a) and a vector of parameters ϕ. Let $R_\phi(s,a)$ be the approximation of R, and assume a similar approximation is used to compute Q_ε (omitted for brevity). If p were known, we could choose $\hat{\phi}$ to minimize the mean-squared TD error given by

$$\hat{\phi} = \underset{\phi}{argmin}\, E\left\{[\vec{r}(s,a) + \beta R_\phi(s',a') - R_\phi(s,a)]^2\right\}, \tag{31}$$

where E denotes the expectation operator over (s,a,s',a'), which requires p. Using results from Tsitsiklis and Roy (1997) on the convergence of TD learning with linear function approximation, AE show that $\hat{\phi}$ can be computed using the *sample expectation operator* E_N in (31) using the formula

$$\hat{\phi} = \left[E_N\{\nu(s,a)[\nu(s,a) - \beta\nu(s',a')]^T\}\right]^{-1} E_N\{\nu(s,a)\vec{r}(s,a)\}. \tag{32}$$

Note that relying on the sample expectation E_N implicitly assumes rational expectations, as it constrains the agent's subjective beliefs to match the empirical data distribution. Using $\hat{\phi}$ the implied estimate $\hat{R}(s,a) = \phi(s,a)\hat{\phi}$ that can be used as the covariate in the CCP estimator of θ in (26), and the other "correction term" $\hat{Q}_\varepsilon(s,a)$ can be estimated in similar fashion. The asymptotic distribution of this two stage TD/CCP estimator of θ is affected by "estimation noise" in the $\hat{R}$ and $\hat{Q}_\varepsilon$ functions that can introduce bias and noise. To deal with this, AE propose a third estimation stage that uses a modified score (i.e. gradient) of the partial log-likelihood (27) to create a robust (Neyman orthogonal) version of the score function (Chernozhukov et al., 2022), obtaining an improved estimator $\hat{\theta}$ robust to first stage estimation noise in $\hat{R}$ and $\hat{Q}_\varepsilon$ by setting the modified score to zero.[21]

The GLADIUS estimator (Kang et al., 2025) (Gradient-based Learning with Ascent-Descent for Inverse Utility learning from Samples) is 1) model-free, 2) does not require first stage non-parametric estimation of $\pi(a|s)$, and 3) does not require a parametric specification for rewards. GLADIUS is an iterative algorithm that uses gradient ascent/descent to maximize a penalized likelihood function similar to the SEES estimator (29) but instead of specifying r as a function of "structural parameters" θ GLADIUS parameterizes the Q function as $Q_{\phi_1}(s,a)$ and the conditional expectation of the value function $E\{V(s')|s,a\}$ as $EV_{\phi_2}(s,a)$ using flexible approximators such as deep neural networks (DNNs). Given estimates $\hat{\phi}_1$ and $\hat{\phi}_2$, GLADIUS recovers the reward function

[21] See Khwaja and Srivastava (2025) for an alternative approach to AE's TD methods that they call RLTD-CCS that combines the TD algorithm from RL with the Conditional Choice Simulator (CCS) estimator of Hotz et al. (1994) that relies on forward simulations of paths to produce a Monte Carlo estimate the $Q_\theta(s,a)$ and hence is a model-based approach to estimation.

as

$$r(s,a) = Q_{\hat{\theta}_1}(s,a) - \beta E V_{\hat{\theta}_2}(s,a), \tag{33}$$

using the Bellman equation for the smoothed Q function, (16).

GLADIUS is similar to SEES in maximizing a penalized likelihood where the penalty term involves a "mean squared Bellman error" for the Q functions, where $Q = \Lambda_\sigma(Q)$ is the fixed point to the smoothed Bellman operator Λ_σ, so Q minimizes the quantity $\rho_{BE}(Q) = E\{[Q(s,a) - \Lambda_\sigma(Q)(s,a)]^2\}$ over the distribution of (s,a) pairs. Unlike SEES (which requires knowledge of p to compute the Bellman error $\rho_{BE}(Q)$), GLADIUS is a model-free approach inspired by Q-learning that uses transitions (s', s, a) in the data to estimate the mean squared Bellman error $\rho_{BE}(Q)$ by appproximating

$$\Lambda_\sigma(Q)(s,a) = r(s,a) + \beta \sum_{s'} V(s') p(s'|a,d) \tag{34}$$

by averaging sample realizations of

$$\tilde{\Lambda}_\sigma(Q)(s',s,a) = r(s,a) + \beta V(s'). \tag{35}$$

This suggests estimating Q by searching for parameters ϕ_1 that minimize the mean squared temporal difference error $\rho_{TD}(Q)$ given by

$$\rho_{TD}(Q_{\phi_1}) = E\{[\Lambda_\sigma(Q_{\phi_1})(s',s,a) - Q_{\phi_1}(s,a)]^2\}, \tag{36}$$

where $\tilde{\Lambda}_\sigma(Q)(s',s,a) - Q(s,a)$ is the TD error. However Kang et al. (2025) note an important complication: the true fixed point Q will minimize the mean squared Bellman error $\rho_{BE}(Q)$ but it will not necessarily minimize the mean squared TD error. To see, this, the note the following identity

$$\rho_{TD}(Q) = \rho_{BE}(Q) + \beta^2 E\{[V(s') - EV(s,a)]^2\}, \tag{37}$$

where the second term reflects expected "heteroscedasticity" in the TD error, i.e. it is the average of the conditional variances given by $H(V)(s,a) = E\{[V(s') - EV(s,a)]^2|s,a\}$. Since V is a function of Q from (17), we write V as $V(Q)$ and the heteroscedasticity term as $H(Q)$. It it follows from (37) that the Q that minimizes $\rho_{TD}(Q)$ will not necessarily be the same as the Q that minimizes $\rho_{BE}(Q)$. To deal with this issue, they express the mean squared Bellman error as $\rho_{BE}(Q) = \rho_{TD}(Q) - H(Q)$. Though $H(Q)$ is also an unknown quantity Kang *et. al.* note that it can be approximated as the solution to

$$H(Q)(s,a) = \min_{v \in R} \sum_{s'} [V(Q)(s') - v]^2 p(s'|s,a), \tag{38}$$

and the v that minimizes (38) is $EV(s,a)$. Using this insight, GLADIUS finds parameters $(\hat{\phi}_1, \hat{\phi}_2)$ to maximize a penalized likelihood function similar to SEES, except it is a max-min problem given

by

$$(\hat{\phi}_1, \hat{\phi}_2) = \underset{\phi_1}{argmax}\ \underset{\phi_2}{argmin} \log(L(\phi_1)) - \lambda\rho(\phi_1, \phi_2), \tag{39}$$

where λ is a penalization parameter, and $\rho(\phi_1, \phi_2)$ is the estimate of the mean squared Bellman error given by

$$\rho(\phi_1, \phi_2) = \frac{1}{N}\sum_{i}^{N}[\Lambda_\sigma(Q_{\phi_1}(s_i', s_i, a_i) - Q_{\phi_1}(s_i, a_i)]^2 - \beta^2[V(Q_{\phi_1})(s_i') - EV_{\phi_2}(s_i, a_i)]^2 \tag{40}$$

Computation of the penalty term can be further reduced to limiting the summation to a subset of states s_i where the corresponding action a_i equals the *normalizing action* because Kang et al. (2025) impose the identifying assumption that for each s there is an action a_s called the normalizing action for which $r(s, a_s)$ is known. From our discussion of identification in section 3.3, this restriction on rewards implies that we can determine all Q values by enforcing the fixed point constraint $Q = \Lambda_\sigma(Q)$ only for the subset of pair (s, a_s) which enables us to determine the corresponding Q at the normalizing action, $Q(s, a_s)$ for all $s \in S$. Then given $Q(s, a_s)$ the remaining $Q(s, a)$ values are determined from $\pi(a|s)$ using the Hotz-Miller inversion formula (30).

We have provided a very selective survey of the IRL literature, focusing on two recent contributions that are closely related to DDC and offer promising new methods for structural estimation. We have emphasized the distinction between model-based and model-free estimation methods, but we wish to make clear that not all IRL methods are model-free. For example Zeng et al. (2024) use non-parametric methods to estimates p to provide a "generative world model" in a model-based version of IRL and "Through extensive experiments, we demonstrate that our algorithm outperforms existing benchmarks for offline IRL and Imitation Learning, especially on high-dimensional robotics control tasks." Thus, it is not obvious that model-free approaches are necessarily superior to model-based approaches, though we agree that good models of p should be used, since "inaccurate models of the world obtained from finite data with limited coverage could compound inaccuracy in estimated rewards." It is also clear that if we are interested in doing a counterfactual that involves changing p, model-based methods will probably be necessary to estimate/construct this new p and then train the model to calculate the new optimal decision rule $\pi(a|s)$ corresponding to counterfactual p.

Many IRL variants that appear distinct prove closely related upon examination. Ziebart et al. (2010)'s maximum causal entropy IRL, for instance, is nearly isomorphic to DDC models under the AS/CI/EV assumptions discussed in section 3. As Gleave and Toyer (2022) notes: "algorithms based on MCE IRL have scaled to high-dimensional environments. Maximum Entropy Deep IRL Wulfmeier et al. (2016) was one of the first extensions, and is able to learn rewards in gridworlds from pixel observations. More recently, Guided Cost Learning Abbeel (2016) and AIRL have scaled to continuous control tasks." Large-scale applications include Barnes et al. (2024) using IRL to infer route preferences from Google Maps data and Zhao and Liang (2023) applying AIRL to

taxi trajectories. Liang et al. (2025) address the opacity of Deep IRL by introducing a post-hoc interpretation framework based on knowledge distillation. They train a surrogate Multinomial Logit (MNL) model using the "soft labels" (probability distributions) generated by the Deep AIRL policy, rather than ground-truth data. This effectively projects the high-dimensional, non-linear behavior learned by the neural network onto a linear-in-parameters structural model, allowing for the recovery of interpretable preference parameters typically associated with DDC. Lee et al. (2026) extend AIRL to unstructured text data.

5 Conclusion

A challenging area for further work is to relax and test rationality assumptions. If decision makers are only "boundedly rational," rewards inferred from observed behavior will be distorted by our attempt to rationalize behavior that may not be fully rational.

The "model-free" approach to inferring rewards (e.g., Adusumilli and Eckhardt (2025); Kang et al. (2025)) avoids specifying transition dynamics p, but implicitly assumes rational expectations: substituting realized transitions for explicit integration effectively ties the agent's subjective beliefs to the data-generating process. When agents hold distorted beliefs (see the tennis example in Section 3), model-free estimators cannot distinguish misspecified rewards from misspecified beliefs. Model-based methods retain the flexibility to test for such distortions, though at the cost of requiring a correctly specified transition model.

We conclude that the limitations confronting DDC and IRL are not computational but rather inherent to inferring rewards and beliefs from data on states and actions, as noted in our discussion of the identification problem in section 2.4. The solution to the identification problem requires us to impose strong prior restrictions (assumptions) about rewards and beliefs.

If these assumptions are wrong, counterfactual predictions from these models may not be accurate even though the models may provide a good fit to the demonstration data used to estimate them. However practical experience suggests that even if our assumptions may not be precisely correct and human decision makers may not be fully rational, the assumptions used to identify DDC and IRL models appear to be sufficiently good approximations to produce realistic counterfactual simulations and leverage limited human demonstration data to train artificially intelligent agents to perform well in tasks previously requiring human expertise. The rapid growth in the DDC and IRL literatures suggests that despite the limitations and challenges we have noted, these methods have enormous practical value. We are likely to see continued rapid progress enabling these methods to be even more widely used to transform economics, finance and artificial intelligence.

References

Abbeel, Chelsea Finn Sergey Levine Pieter, "Guided Cost Learning: Deep Inverse Optimal Control via Policy Optimization," *arXiv*, 2016, *arXiv:1603.00448v3.*

Abbring, Jaap and Øystein Daljord, "Identifying the discount factor in dynamic discrete choice models," *Quantitative Economics*, 2020, *11* (2), 471–501.

Adams, Stephen, Tyler Cody, and Peter A. Beling, "A survey of inverse reinforcement learning," *Artificial Intelligence Review*, 2022, *55*, 4307–4436.

Adusumilli, Kaun and Dita Eckhardt, "Temporal Difference Estimation of Dynamic Discrete Choice Models," *Review of Economic Studies*, 2025, *forthcoming.*

Aguirregabiria, Victor and Pedro Mira, "Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models," *Econometrica*, 2002, *70*, 1519–1543.

_ **and** _ , "Dynamic discrete choice structural models: A survey," *Journal of Econometrics*, 2010, *156*, 38–67.

Anderson, Axel, Jeremy Rosen, John Rust, and Kin-Ping Wong, "Disequilibrium Play in Tennis," *Journal of Political Economy*, 2025, *133* (1), 190–251.

Arora, Saurabh and Prashant Doshi, "A survey of inverse reinforcement learning: Challenges, methods and progress," *Artificial Intelligence*, 2021, *297* (103500).

Bajari, Patrick, C. Lanier Benkard, and John Levin, "Estimating Dynamic Models of Imperfect Competition," *Econometrica*, 2007, *75* (5), 1331–1370.

Barnes, M., M.Abueg, O. F. Lange, M. Deeds, J. Trader, D. Molitor, M. Wulfmeier, and S. O'Banion, "Massively Scalable Inverse Reinforcement Learning in Google Maps," *OpenReview.net*, 2024, pp. 1–22.

Barzegary, Ebrahim and Hema Yoganarasimhan, "A recursive partitioning approach for dynamic discrete choice modeling in high dimensional settings," *arXiv*, 2022, *arXiv:2208.01476.*

Bellman, Richard, *Dynamic Programming*, 4th ed., Princeton University Press, 1957.

_ **and Stuart Dreyfus**, "Functional Approximations and Dynamic Programming," *Mathematical Tables and Other Aids to Computation*, 1959, *13* (68), 247–251.

Bray, Robert L., "A Comment on 'Using Randomization to Break the Curse of Dimensionality'," *Econometrica*, 2022, *90* (4), 1915–1929.

Chernozhukov, Victor, Juan Carlos Escanciano, Hidehiko Ichimura, Whitney K. Newey, and James M. Robins, "Locally Robust Semiparametric Estimation," *Econometrica*, 2022, *90* (4), 1051–1535.

Chow, C. S. and J. N. Tsitsiklis, "The Complexity of Dynamic Programming," *Journal of Complexity*, 1989, *5*, 466–488.

Christiano, Paul F, Jan Leike, Tom B Brown, Miljan Martic, Shane Legg, and Dario Amodei, "Deep reinforcement learning from human preferences," in "Advances in Neural Information Processing Systems," Vol. 30 2017.

Dearing, Adam and Jason Blevins, "Efficient and Convergent Sequential Pseudo-Likelihood Estimation of Dynamic Discrete Choice Games," *Review of Economic Studies*, 2025, *92*, 981–1021.

Ermon, Stefano, Yexiang Xue, Russell Toth, Bistra Dilkina, Richard Bernstein, Theodoros Damoulas, Patrick Clark, Steve DeGloria, Andrew Mude, Christopher Barrett, and Carla P. Gomes, "Learning large-scale dynamic discrete choice models of spatio-temporal preferences with application to migratory pastoralism in East Africa," *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015, p. 644–650.

Fu, Justin, Aviral Kumar, Matthew Soh, and Sergey Levine, "Diagnosing Bottlenecks in Deep Q-learning Algorithms," in "Proceedings of the 36th International Conference on Machine Learning" PMLR 2019, pp. 2021–2030.

_ , Katie Luo, and Sergey Levine, "Learning Robust Rewards with Adversarial Inverse Reinforcement Learning," in "International Conference on Learning Representations (ICLR)" 2018.

Fujimoto, Scott, David Meger, Doina Precup, Ofir Nachum, and Shixiang Shane Gu, "Why Should I Trust You, Bellman? The Bellman Error is a Poor Replacement for Value Error," in "Proceedings of the 39th International Conference on Machine Learning" PMLR 2022, pp. 6907–6935.

Ghosal, Gaurav R., Matthew Zurek, Daniel S. Brown, and Anca D. Dragan, "The Effect of Modeling Human Rationality Level on Learning Rewards from Multiple Feedback Types," in "Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence" 2023, pp. 5983–5992.

Gillingham, Kenneth, Fedor Iskhakov, Anders Munk-Nielsen, John Rust, and Bertel Schjerning, "Equilibrium Trade in Automobiles," *Journal of Political Economy*, 2022, *130* (10), 2534–2593.

Gleave, Adam and Sam Toyer, "A Primer on Maximum Causal Entropy Inverse Reinforcement Learning," *arXiv*, 2022, *arXiv:2203.11409v1.*

Haarnoja, Tuomas, Haoran Tang, Pieter Abbeel, and Sergey Levine, "Reinforcement Learning with Deep Energy-Based Policies," *Proceedings of the 34th International Conference on Machine Learning*, 2017.

Ho, Jonathan and Stefano Ermon, "Learning large-scale dynamic discrete choice models of spatio-temporal preferences with application to migratory pastoralism in East Africa," *Advances in neural information processing systems*, 2016, *29.*

Hotz, Joseph V. and Robert A. Miller, "Conditional choice probabilities and the estimation of dynamic models," *Review of Economic Studies*, 1993, *60*, 497–529.

_ , _ , Seth Sanders, and Jeffrey Smith, "A simulation estimator for dynamic models of discrete choice," *Review of Economic Studies*, 1994, *61*, 265–289.

Howard, Ronald A., *Dynamic Programming and Markov Processes*, Technology Press, Cambridge, MA, 1960.

Imai, Susumu, Neelam Jain, and Andrew Ching, "Bayesian Estimation of Dynamic Discrete Choice Models," *Econometrica*, 2009, *77* (6), 1865–1899.

Iskhakov, Fedor, Jinhyuk Lee, John Rust, Bertel Schjerning, and Kyoungwon Seo, "Constrained Optimization Approaches to Estimation of Structural Models: Comment," *Econometrica*, 2016, *84* (1), 365–370.

Jiang, Nan and Tengyang Xie, "Offline Reinforcement Learning in Large State Spaces: Algorithms and Guarantees," *manuscript, University of Illinois*, 2024.

Kaji, Tetsuya, Elena Manresa, and Guillaume Pouliot, "An Adversarial Approach to Structural Estimation," *Econometrica*, 2023, *91* (6), 2041–2063.

Kalouptsidi, Myrto, Paul T. Scott, and Eduardo Souza-Rodrigues, "Identification of counterfactuals in dynamic discrete choice models," *Quantitative Economics*, 2021, *12* (2), 351–403.

Kang, Enoch H., Hema Yoganarasimhan, and Lalit Jain, "An Empirical Risk Minimization Approach for Offline Inverse RL and Dynamic Discrete Choice Model," *arXiv*, 2025, *arXiv:2502.14131.*

Khwaja, Ahmed and Sonal Srivastava, "Reinforcement Learning Based Computationally Faster Conditional Choice Simulation Estimation of Dynamic Discrete Choice Models," *manuscript, Judge Business School, University of Cambridge*, 2025.

Lee, Peter S., K. Sudhir, and Tong Wang, "Modeling Serialized Content Consumption: Adversarial IRL for Dynamic Discrete Choice," 2026. Job Market Paper, Yale School of Management.

Liang, Yuebing, Shenhao Wang, Jiangbo Yu, Zhan Zhao, Jinhua Zhao, and Sandy Pentland, "Analyzing sequential activity and travel decisions with interpretable deep inverse reinforcement learning," *Travel Behaviour and Society*, 2025, *43*, 101171.

Lucas, Jr. Robert E., "Econometric Policy Evaluation: A Critique," in Alan Meltzer, ed., *The Phillips Curve and Labor Markets. Carnegie-Rochester Conference Series on Public Policy. Vol. 1*, Elsevier, 1976, pp. 19–46.

Luo, Yao and Peijun Sang, "Efficient Estimation of Structural Models via Sieves," *arXiv*, 2024, *arXiv:2204.13488v2.*

Magnac, Thierry and David Thesmar, "Identifying Dynamic Discrete Decision Processes," *Econometrica*, 2002, *70*, 801–816.

McFadden, Daniel L., "Conditional Logit Analysis of Qualitative Choice Behavior," in P. Zarembka, ed., *Frontiers in Econometrics*, Academic Press, 1973, pp. 105–142.

Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and

D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, 2015, *518*, 518–533.

Ng, Andrew Y. and Stuart Russell, "Algorithms for Inverse Reinforcement Learning," *Proceedings of the Seventeenth International Conference on Machine Learning*, 2000, p. 663–670.

_ , Daishi Harada, and Stuart Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," in "Proceedings of the Sixteenth International Conference on Machine Learning" 1999, pp. 278–287.

Nguyen, Hoang, "Neural Networks for Efficient Estimation of High-Dimensional Dynamic Discrete Choice Models," *manuscript, Georgetown University*, 2025.

Ramachandran, Deepak and Eyal Amir, "Bayesian Inverse Reinforcement Learning," *Proceedings IJCAI*, 2007, p. 2586–2591.

Robbins, H. and S. Munro, "A Stochastic Approximation Method," *Annals of Statistics*, 1951, *22*, 400–407.

Rust, John, "Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher," *Econometrica*, 1987, *55* (5), 993–1033.

_ , "Maximum Likelihood Estimation of Discrete Control Processes," *SIAM Journal on Control and Optimization*, 1988, *26* (5), 1006–1024.

_ , "Structural Estimation of Markov Decision Processes," in R. Engle and D. McFadden, eds., *The Handbook of Econometrics, Volume 4*, Elsevier, 1994, pp. 3082–3139.

_ , "Using Randomization to Break the Curse of Dimensionality," *Econometrica*, 1997, *65* (3), 487–516.

_ , "The Limits to Inference *with* Theory: A Review of Wolpin (2013)," *Journal of Economic Literature*, 2014, *52* (3), 820–850.

Silver, David, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, 2018, *362*, 1140–1144.

Su, Che-Lin and Kenneth L. Judd, "Constrained Optimization Approaches to Estimation of Structural Models," *Econometrica*, 2012, *80* (5), 2213–2230.

Sutton, Richard J. and Andrew G. Barto, *Reinforcement Learning An Introduction*, 2nd ed., MIT Press, 2020.

Sutton, Richard S., "Learning to Predict by the Methods of Temporal Differences," *Machine Learning*, 1988, *3*, 9–44.

_ **and Andrew G. Barto**, *Reinforcement Learning An Introduction*, 1st ed., MIT Press, 1998.

Tsitsiklis, John N., "Asynchronous Stochastic Approximation and Q-Learning," *Machine Learning*, 1994, *16*, 185–202.

_ **and Benjamin Van Roy**, "An analysis of temporal-difference learning with function approximation," *IEEE transactions on automatic control*, 1997, *42*, 674–690.

Wang, Siyue, Zhaorun Chen, Zhuokai Zhao, Chaoli Mao, Yiyang Zhou, Jiayu He, and Albert Sibo Hu, "ESCIRL: Evolving Self-Contrastive IRL for Trajectory Prediction in Autonomous Driving," *8th Conference on Robot Learning (CoRL 2024)*, 2025, pp. 1–20.

Watkins, C. J. C. H., *Learning from Delayed Rewards*, PhD Thesis, Kings College, Cambridge UK, 1989.

Wulfmeier, Markus, Peter Ondrúška, and Ingmar Posner, "Maximum Entropy Deep Inverse Reinforcement Learning," *arXiv*, 2016, *arXiv:1507.04888v3.*

Yudin, D. B. and A. B. Nemirovsky, "Estimating the Computational Complexity of Mathematical Programming Problems," *Ekonomika i Matem. Metody*, 1976, *12*, 128–142.

Yue, Sheng, Guanbo Wang, Wei Shao, Zhaofeng Zhang, Sen Lin, Ju Ren, and Junshan Zhang, "CLARE: Conservative Model-Based Reward Learning for Offline Inverse Reinforcement Learning," in "The Eleventh International Conference on Learning Representations" 2023.

Zeng, Siliang, Chenliang Li, Alfredo Garcia, and Mingyi Hong, "When Demonstrations Meet Generative World Models: A Maximum Likelihood Framework for Offline Inverse Reinforcement Learning," *37th Conference on Neural Information Processing Systems*, 2023.

_ **, Mingyi Hong, and Alfredo Garcia**, "Structural Estimation of Markov Decision Processes in High-Dimensional State Space with Finite-Time Guarantees," *Operations Research*, 2024, *73* (2).

Zhao, Zhan and Yuebing Liang, "A deep inverse reinforcement learning approach to route choice modeling with context-dependent rewards," *Transportation Research Part C*, 2023, *149.*

Ziebart, Brian D., Andrew Maas, J. Andrew Bagnell, and Anind K. Dey, "Maximum Entropy Inverse Reinforcement Learning," *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, 2008, *55* (5), 1433–1438.

_ **, J. Andrew Bagnell, and Anind K. Dey**, "Modeling Interaction via the Principle of Maximum Causal Entropy," *Proceedings of the 27th International Conference on Machine Learning*, 2010.